

Introduction to Inter-Process Communication (IPC)

Inter-Process Communication (IPC) refers to the mechanism that allows **two or more processes to communicate and coordinate** while executing simultaneously.

Processes executing concurrently are of two types:

(i) Independent Processes

- Do not depend on the execution of other processes.
- Do not share data with other processes.
- Execution of one does not affect another.

(ii) Co-operating Processes

- Can affect or be affected by other processes.
- Share data with other processes.
- Require synchronization during execution.

Why We Need IPC

Processes cooperate for the following reasons:

1. **Information Sharing**
Allows multiple processes to access common data.
2. **Computation Speed-up**
Dividing tasks among processes improves performance.
3. **Modularity**
Large programs are divided into smaller processes.
4. **Convenience**
Multiple activities can be performed at the same time.

Race Condition

A **race condition** occurs when:

Two or more processes access and modify shared data at the same time and the final result depends on the order of execution.

Race condition leads to **inconsistent and incorrect results**.

Reasons for Race Condition

1. Instruction execution order is unpredictable.
2. Sharing of resources such as files, memory, or data.

Example of Race Condition

Consider two threads incrementing a shared variable.

Both read the same value → both increment → both write back.

Instead of value increasing twice, it increases only once, leading to **wrong output**.

This happens because operations are **not mutually exclusive**.

Assume that two threads each increment the value of a global integer variable by 1. Ideally, the following sequence of operations would take place:

Thread 1	Thread 2	Integer value
		0
read value		← 0
increase value		0
write back		→ 1
	read value	← 1
	increase value	1
	write back	→ 2

In the case shown above, the final value is 2, as expected. However, if the two threads run simultaneously without locking or synchronization, the outcome of the operation could be wrong. The alternative sequence of operations below demonstrates this scenario:

Thread 1	Thread 2	Integer value
		0
read value		← 0
	read value	← 0
increase value		0
	increase value	0
write back		→ 1
	write back	→ 1

In this case, the final value is 1 instead of the correct result of 2. This occurs because here the increment operations are not mutually exclusive. Mutually exclusive operations are those that cannot be interrupted while accessing some resource such as a memory location.

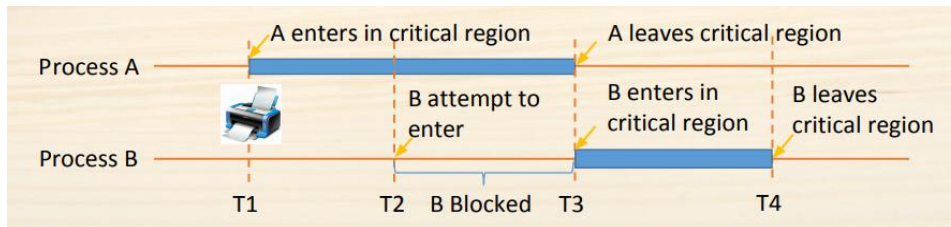
Critical Section

The **critical section** is the part of a program where **shared resources or variables are accessed**.

Only **one process at a time** is allowed inside the critical section.

If more than one process enters simultaneously, inconsistency occurs.

Hence, the critical section is **protected** to avoid errors.



T1: Process A enters critical region

- Process A starts executing its **critical section**.
- The shared resource (printer in the diagram) is now **locked by A**.
- No other process can enter the critical section at this time.

T2: Process B attempts to enter

- Process B tries to enter the critical section.
- But since A is already inside, **B is blocked (waiting state)**.
- This ensures **mutual exclusion** — only one process at a time.

This waiting period is shown as “**B Blocked**”

T3: Process A leaves, B enters

- Process A finishes its work and **exits the critical section**.
- Now the resource becomes free.
- Immediately, Process B is allowed to **enter the critical section**.

T4: Process B leaves

- Process B completes its task and **leaves the critical section**.
- Now the resource is free again for any other process.

Key Concepts Demonstrated

1. Mutual Exclusion

- Only one process (A or B) is inside the critical section at a time.

2. Blocking / Waiting

- Process B waits when A is using the resource.

3. Proper Synchronization

- No overlap → prevents **race conditions**.

Need of Critical Section

Suppose:

- Process A reads variable x.
- Process B writes variable x at the same time.

If Process A needs the updated value, simultaneous execution may give incorrect results.

Therefore:

- Variable x is protected.
- One process enters first.
- After completion, the other enters.

This avoids inconsistent output.

Process A:

```
// Process A
.  
.  
b = x + 5;           // instruction executes at time = Tx  
.
```

Process B:

```
// Process B
.  
.  
x = 3 + z;           // instruction executes at time = Tx  
.
```

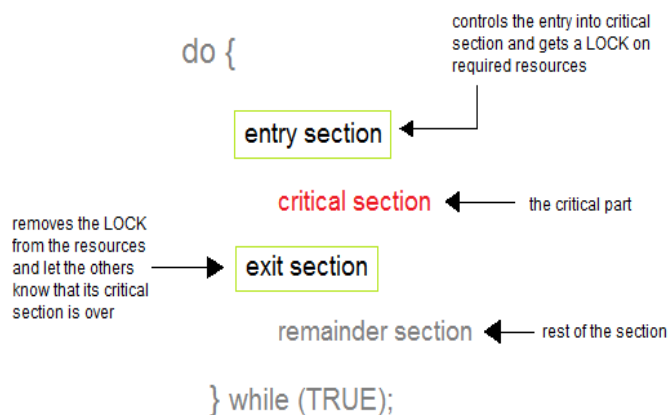
Solution to Critical Section Problem

A simple solution is to use a **Brute-Force approach by using locking mechanism.**

A process must:

- Enter the **entry section** and acquire lock.
- Execute the **critical section**.
- Leave through **exit section** and release lock.

The remaining code is called the **remainder section**.



Program parts:

1. **Entry Section** – requests permission to enter critical section
2. **Critical Section** – accesses shared data
3. **Exit Section** – releases lock
4. **Remainder Section** – remaining code

Only the process that holds the lock can enter the critical section.

Conditions for Correct Critical-Section Solution

A correct solution must satisfy:

(i) Mutual Exclusion

Only one process can be inside the critical section at a time.

(ii) Progress

If no process is in the critical section, one of the waiting processes must be allowed to enter.

(iii) Bounded Waiting

A process cannot wait forever; there is a limit to how long others can enter before it gets chance.

Solutions to Synchronization Problem

Synchronization can be achieved using:

1. **Hardware Solutions**
2. **Software Solutions**
3. **Strict Alternation**

These methods ensure safe access to shared resources.

Hardware Approach – Test-and-Set and Swap

Hardware-based synchronization assumes:

- Exclusive access to memory locations
- Special atomic instructions

Test-and-Set Instruction

It is a hardware instruction that performs two actions **atomically**:

1. Stores the current value
2. Sets the variable to true

This helps in implementing locks.

Swap Instruction

- Atomically exchanges the contents of two memory locations
- Used for synchronization.

Test-and-Set – Program Structure

Using Test-and-Set:

- Process repeatedly checks the lock in entry section
- Enters critical section when lock is free
- Sets lock to release in exit section

This prevents two processes from entering at the same time.

```
int TestAndSet(int &lock) {  
    int initial = lock;  
    lock = 1;  
    return initial;  
}  
  
void enter_CS(X)  
{  
    while test-and-set(X) ;  
}  
  
void leave_CS(X)  
{  
    X = 0;  
}
```

Explanation:

- It reads the current value of lock
- Then sets lock = 1 (locked)
- Returns the old value

Important:

This operation is **atomic** (executed in one step, no interruption allowed)

If lock = 0 (free):

- initial = 0
- lock = 1
- returns 0

If lock = 1 (busy):

- initial = 1
- lock = 1
- returns 1
- Process keeps calling test-and-set(X)
- If result = 1 → someone else is using CS → keep waiting (busy waiting)
- If result = 0 → lock acquired → enter CS
- Sets lock = 0
- Makes CS available for other processes

Peterson's Solution (Introduction)

Peterson's solution is a **software-based synchronization technique** for **two processes**.

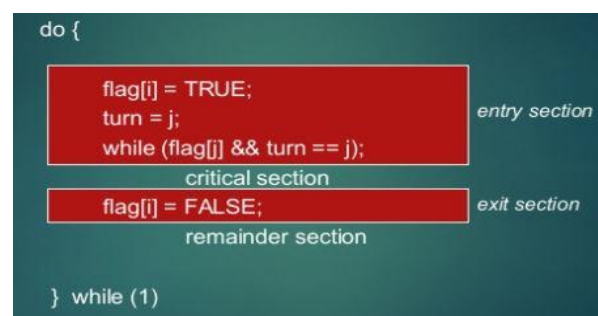
It assumes:

- LOAD and STORE instructions are atomic

Two shared variables are used:

1. **int turn**
 - Indicates whose turn it is to enter the critical section
2. **Boolean flag []**
 - flag[i] = true means process P_i wants to enter the critical section

These variables coordinate entry into the critical section.



Explanation:

1. Variables Used

- **flag []**
 - $\text{flag}[i] = \text{TRUE} \rightarrow \text{Process } i \text{ wants to enter CS}$
 - $\text{flag}[j] = \text{TRUE} \rightarrow \text{Process } j \text{ wants to enter CS}$

It indicates **interest**

- **turn**
 - Decides **whose turn it is**
 - Helps avoid conflict when both processes want to enter

Entry Section (Important)

Step 1: Show Interest

Process i says: “I want to enter CS”

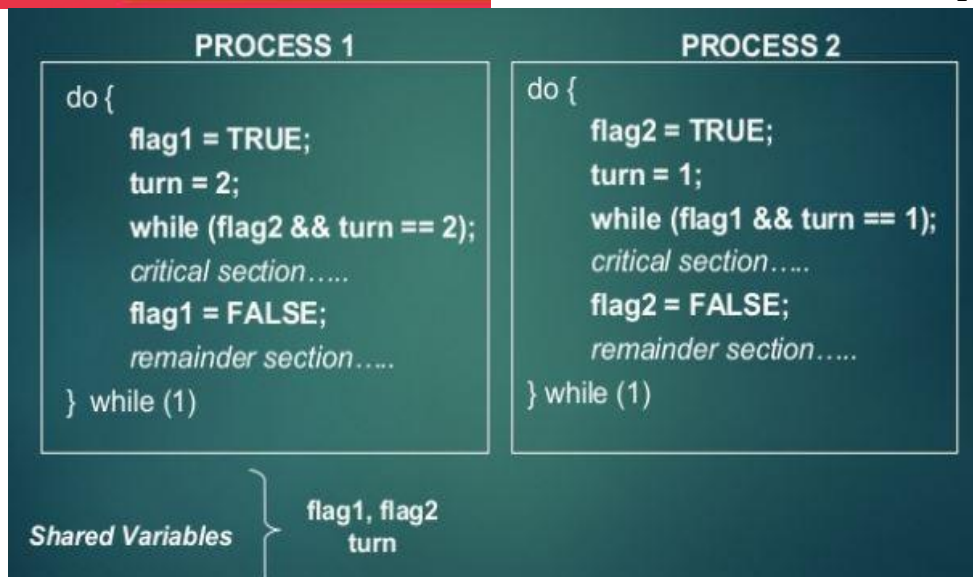
Step 2: Give Chance to Other

Process i says: “If conflict happens, let process j go first”

Step 2: Wait Condition

Wait if:

- Other process also wants to enter ($\text{flag}[j] == \text{TRUE}$)
- AND it's their turn ($\text{turn} == j$)



“Both processes declare interest, and turn ensures only one enters at a time.”

Peterson's Solution – Analysis and Limitations

Properties Satisfied

Peterson's solution ensures:

1. **Mutual Exclusion** – Only one process enters the critical section
2. **Progress** – Waiting processes can proceed
3. **Bounded Waiting** – Every process eventually gets a turn

Disadvantages

1. Involves **busy waiting**
2. Works only for **two processes**

Strict Alternation

Strict Alternation is a software synchronization technique used only for **two processes**.

- Uses a shared variable called **turn**
- A process can enter the critical section only when turn equals its number

- After leaving, it changes the turn to the other process

This method causes **busy waiting** and is limited to two processes.

```
Non - CS
while (turn != i);
Critical Section
turn = j;
Non - CS

Non - CS
while (turn != j);
Critical Section
turn = i;
Non - CS
```

Code 1:

- Process **i** waits until `turn == j` becomes false
i.e., waits until `turn == i`
- Then enters **Critical Section**
 - After finishing → sets `turn = i` (gives next chance accordingly)

Code 2:

- Process **j** waits until `turn == i`
- Then enters **Critical Section**
 - After finishing → sets `turn = j`

Example flow:

1. `turn = i`
 - Process **i** enters CS
 - Process **j** waits
2. After **i** finishes:
 - `turn = j`
3. Now:
 - Process **j** enters CS
 - Process **i** waits

This continues alternately

Producer–Consumer Problem

The Producer–Consumer problem is a classic **multi-process synchronization** problem.

- Also called the **bounded buffer problem**
- Two processes share a fixed-size buffer:
 - **Producer** inserts data
 - **Consumer** removes data

Conditions to Avoid Inconsistency

1. Producer must not insert data when buffer is full
2. Consumer must not remove data when buffer is empty

Producer – Solution

For safe operation:

- Producer sleeps or waits if buffer is full
- When consumer removes an item, producer is notified
- Synchronization tools are used to coordinate access

Consumer – Solution

For safe operation:

- Consumer sleeps if buffer is empty
- When producer adds data, consumer is notified
- Synchronization mechanisms manage communication

	Producer	Consumer
Empty	YES	NO
Filled	NO	YES
Partially filled	YES	YES

SEMAPHORE

A **Semaphore** is an integer variable used to solve the **critical-section problem** and to synchronize processes.

Semaphores work using two **atomic operations**:

1. **Wait ()**
2. **Signal ()**

These operations ensure that only permitted processes can enter the critical section.

wait(S) Operation

The **wait ()** operation:

- Decrements the value of semaphore **S** if it is positive.
- If $S \leq 0$, the process is forced to wait.

Pseudo-code:

```
wait(S)
{
    while (S <= 0);
    S--;
}
```

signal(S) Operation

The **signal ()** operation:

- Increments the value of semaphore **S**.
- Indicates that a resource has been released.

Pseudo-code:

```
signal(S)
{
    S++;
}
```

Basically:

A semaphore is an integer variable used for process synchronization through atomic wait() and signal() operations.

Types of Semaphores

There are two main types:

1. **Counting Semaphore**
2. **Binary Semaphore**

These are used to control access to shared resources.

Counting Semaphore

A **counting semaphore** has an unrestricted integer value.

- The semaphore value represents the **number of available resources**.
- When a resource is allocated → value is decremented.
- When a resource is released → value is incremented.

Use

Counting semaphores are used when multiple identical resources exist.

Binary Semaphore

Binary semaphore is a special type of semaphore whose value is limited to **0 and 1**.

- Works similar to a lock.
- **Wait()** succeeds only when value is 1.

- **signal()** is used to release the lock.

Binary semaphores are easier to implement in many cases.

Monitor – Introduction

A **Monitor** is a high-level synchronization construct.

It is a special module that contains:

- Shared variables
- Procedures that operate on them
- Synchronization logic

Important property:

Only one process can be active inside a monitor at a time.

Processes cannot directly access the internal data of the monitor.

Working of a Monitor

When a process calls a monitor procedure:

- The monitor first checks if another process is already inside.
- If yes → calling process is blocked.
- If no → process enters the monitor.

This ensures **mutual exclusion** automatically.

Producer–Consumer Problem Using Monitor

In monitors, special variables called **condition variables** are used.

Two operations are provided:

- **wait**
- **signal**

If producer finds buffer full:

- It waits on a condition variable.
- Producer is blocked.

This allows another process to enter the monitor.

When consumer removes an item:

- It performs **signal** on the condition variable.
- Producer is awakened.

Rules:

- Only one process is active inside monitor.
- **signal** usually appears as the last statement.
- If many processes wait, only one is awakened by scheduler.

Working

- If the producer finds the buffer full, it executes **wait()** and gets blocked.
- If the consumer finds the buffer empty, it executes **wait()** and gets blocked.
- When the producer inserts an item, it performs **signal()** to wake the consumer.
- When the consumer removes an item, it performs **signal()** to wake the producer.

```
1 monitor Producer_Consumer
2 {
3     int item_Count;
4
5     condition full;
6     condition empty;
7
8
9     procedure add(item)
10    {
11        while (item_Count == Buffer_S)
12        {
13            wait(full);
14        }
15
16        putItemIntoBuffer(item);
17
18        item_count = item_count + 1;
19
20
21        if (item_count == 1)
22        {
23            notify(empty);
24        }
25    }
```

Variables:

- item_count → number of items in buffer
- full → condition when buffer is full
- empty → condition when buffer is empty

Producer Code: add(item)

- If buffer is **full**, producer **waits**
- wait(full) → producer sleeps until space is available

Adding Item

Producer:

- Adds item to buffer
- Increases count

Notify Consumer

- If buffer was **empty before** (0 → 1)
- Now consumer can consume

So:

- notify(empty) wakes up waiting consumer

```
1 Procedure remove()
2 {
3     while (item_Count == 0)
4     {
5         wait(empty);
6     }
7
8     item = removeItemFromBuffer();
9
10    item_Count = item_Count - 1;
11
12    if(item_Count == Buffer_S - 1)
13    {
14        notify(full);
15    }
16 }
17 }
```

1. Check if Buffer is Empty

If no items are available:

- Consumer **cannot consume**
- So it goes to sleep using wait(empty)

It will wake up only when producer adds something

2. Remove Item

Consumer removes an item from buffer

3. Decrease Count

Update number of items in buffer

4. Notify Producer

Buffer was **full before removal**

Now **one space is available**

So, producer (waiting on full) should be notified

Readers–Writers Problem (Introduction)

The **Readers–Writers Problem** is a classical synchronization problem.

It considers a shared resource such as a file or database accessed by many users.

Rules:

- If one user is **writing**, no other user can read or write.
- If users are only **reading**, multiple readers can read simultaneously.

This is required to prevent inconsistency in shared data.

Readers–Writers Problem – Cases

Two types of operations exist:

1. **Read operation**
2. **Write operation**

Users are classified as:

- **Reader**
- **Writer**

Problem situations:

- Reader + Writer → **Problem**
- Writer + Reader → **Problem**

These cases may lead to inconsistent data.

Readers–Writers Problem – All Conditions

Four cases must be handled:

1. Reader + Reader → **No problem**
2. Reader + Writer → **Problem**
3. Writer + Reader → **Problem**
4. Writer + Writer → **Problem**

Synchronization is required to control these situations.

Semaphores are commonly used to solve this problem.

```
1 int read_count = 0
2
3 semaphore mutex = 1
4 semaphore b = 1
5
6 void reader(void)
7 {
8     while(true)
9     {
10         down(mutex);
11
12         read_count = read_count+1;
13         if(read_count== 1) then down(b)
14
15         up(mutex)
16
17         readDataBase();
18
19         down(mutex)
20
21         read_count =read_count-1;
22
23         if(read_count== 0)then up (b)
24         up(mutex);
25
26         use_read_data();
27     }
28 }
29
```

Reader Code Explained

1. Entry Section

What happens:

- First reader locks mutex
- Increments read_count
- If **first reader (read_count == 1)**:
Locks b → blocks writers
- Releases mutex

Ensures:

- First reader blocks writers
- Other readers can enter freely

2. Reading Section

Multiple readers can execute this together

3. Exit Section

- ☐ Reader decreases count
- ☐ If **last reader leaves** (**read_count == 0**):
Releases b → allows writers
- ☐ Releases mutex

4. Remaining Work

Reader uses the data (outside critical section)

Case 1: First Reader

- Locks b → writers blocked

Case 2: More Readers

- Directly enter (no waiting)

Case 3: Last Reader Leaves

- Releases b → writers allowed

```
1  Void write(void)
2  {
3      while(true)
4      {
5          down(b);
6
7          readDataBase();
8      }
9  }
```

Writer part:

1. Request Access

Writer tries to access the shared resource

- If b = 1 → writer enters
- If b = 0 → writer waits

b ensures **exclusive access**

2. Writing Section

Writer modifies the shared data

No reader or other writer allowed at this time

3. Release Resource

Writer releases the resource

Now readers or other writers can proceed

Case 1: No Readers

- $b = 1$
- Writer enters directly

Case 2: Readers Present

- First reader already did $\text{down}(b)$
 - So $b = 0$
- Writer must wait

Case 3: Last Reader Leaves

Last reader does $\text{up}(b)$
Writer gets chance

Dining Philosophers Problem (Introduction)

The **Dining Philosophers Problem** is another classical synchronization problem.

Five philosophers sit around a circular table.

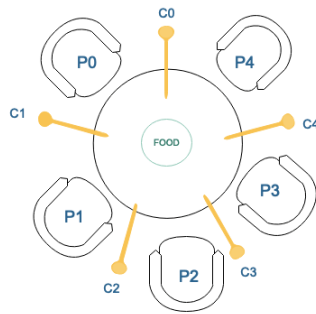
Between each pair is one chopstick.

Rules:

- A philosopher must pick **two chopsticks** (left and right) to eat.
- A philosopher can either be:
 - Thinking

- Eating
- Chopsticks must be released after eating.

If not controlled properly, deadlock may occur.



Chopstick Requirement

In the Dining Philosophers problem:

- Five philosophers (P0–P4) sit around a circular table.
- Five chopsticks (C0–C4) are placed between them.
- Each philosopher needs **two chopsticks** to eat:
 - One from the left
 - One from the right

For philosopher P_i :

- Left chopstick = C_i
- Right chopstick = $C_{(i+1) \bmod N}$

A philosopher alternates between:

- Thinking
- Eating

Dining Philosophers – Normal Case

If one philosopher finishes eating:

- He puts down both chopsticks.
- The next philosopher can use the released chopsticks.

- In this case, no conflict occurs.

This represents the **best case**, where philosophers eat one by one peacefully.

```
1 void philisopher (void)
2 {
3     while(true)
4     {
5         thinking();
6
7         take_chopstick(i);
8         take_chopstick((i+1) % N);
9         eat();
10
11        put_chopstick(i);
12        put_chopstick((i+1) % N);
13    }
14 }
```

Thinking

philosopher is not using any resource

Pick Left Chopstick

Takes chopstick number **i** (left)

Pick Right Chopstick

Takes right chopstick
(i+1) % N ensures circular arrangement

Eating

Needs **both chopsticks**

Put Back Chopsticks

Releases both chopsticks

$(i+1) \% N$?

Ensures:

- Last philosopher connects back to first
- Circular structure

Dining Philosophers – Conflict Situation

If two neighbouring philosophers try to eat at the same time:

- They may need the same chopstick.
- If the chopstick is already taken, the second philosopher must wait.

To control this situation:

- **Binary semaphores** are used for each chopstick.
- A philosopher is blocked if a required chopstick is unavailable.

Dining Philosophers – Semaphore Assignment

Each chopstick is represented by a semaphore:

- S_0, S_1, S_2, S_3, S_4

Each philosopher requires two semaphores:

- $P_0 \rightarrow S_0 \text{ and } S_1$
- $P_1 \rightarrow S_1 \text{ and } S_2$
- $P_2 \rightarrow S_2 \text{ and } S_3$
- $P_3 \rightarrow S_3 \text{ and } S_4$
- $P_4 \rightarrow S_4 \text{ and } S_0$

Initially, all semaphore values are **1**, meaning chopsticks are free.

```
1 void philisopher (void)
2 {
3     while(true)
4     {
5         thinking();
6
7         wait(take_chopstick(Si));
8         wait(take_chopstick((Si+1) % N));
9         eat();
10
11         signal( put_chopstick(i));
12         signal( put_chopstick((i+1) % N));
13     }
14 }
15
16
17
18
```

Each **chopstick** is a **semaphore**

- Value = 1 → available
- Value = 0 → in use
-

1. Thinking

Philosopher is idle (no resource used)

2. Pick Chopsticks (wait)

wait() = **down()** operation

- If chopstick is free → take it
- If not → wait

Needs **both** chopsticks to eat

3. Eating

Happens only when both chopsticks are acquired

4. Release Chopsticks (signal)

signal() = **up()** operation

- Releases chopsticks
- Allows others to use them

Even with semaphores, this **still has DEADLOCK problem**

Deadlock Scenario:

1. All philosophers do wait(chopstick[i])
2. Everyone picks left chopstick
3. Everyone waits for right chopstick
4. No one releases → **deadlock**

Dining Philosophers – Semaphore State Example

When philosopher P0 is eating:

- Semaphores S0 and S1 become **0**
- Other philosophers needing S0 or S1 must wait

This ensures that no two philosophers use the same chopstick at the same time.

Each chopstick is a **binary semaphore (0 or 1)**

Ensures **mutual exclusion** (only one philosopher can use a chopstick)

Dining Philosophers – Deadlock Situation

If all philosophers simultaneously pick their first chopstick:

- Each holds one chopstick
- No one can get the second

This results in **deadlock**, where:

- All processes wait forever
- No progress is possible

Deadlock is an undesirable system state.

S0	S1	S2	S3	S4
1	1	1	1	1
↓	↓			
0	0			

Dining Philosophers – Deadlock Avoidance Idea

To avoid deadlock:

- One philosopher is forced to pick chopsticks in **reverse order**.
- For example, philosopher P4 first tries to pick S4, not S0.
- This breaks circular waiting.

As a result:

- One philosopher succeeds
- Releases chopsticks
- Others proceed one by one

P0	S0,S1
P1	S1,S2
P2	S2,S3
P3	S3,S4
P4	S4,S0

P0	S0,S1
P1	S1,S2
P2	S2,S3
P3	S3,S4
P4	S0,S4



PARUL INSTITUTE OF ENGINEERING &
TECHNOLOGY
FACULTY OF ENGINEERING & TECHNOLOGY
PARUL UNIVERSITY

S0	S1	S2	S3	S4
0	0	0	0	1
S0	S1	S2	S3	S4
0	0	0	1	1
S0	S1	S2	S3	S4
0	0	1	1	1